===== file: entities/FinancialsHistory =====

json

```
{
  "name": "FinancialsHistory",
  "type": "object",
  "properties": {
    "ticker": { "type": "string", "description": "Stock ticker symbol" },
    "period_end": { "type": "string", "format": "date", "description": "End date
of the financial period (YYYY-MM-DD)" },
    "freq": { "type": "string", "enum": ["annual", "quarterly"], "description":
"Reporting frequency" },
    "revenue": { "type": "number", "description": "Total revenue" },
    "cost_of_revenue": { "type": "number", "description": "Cost of revenue/goods
sold" },
    "gross_profit": { "type": "number", "description": "Gross profit" },
    "operating_income": { "type": "number", "description": "Operating income" },
    "ebit": { "type": "number", "description": "Earnings before interest and
taxes" },
    "net_income": { "type": "number", "description": "Net income" },
    "research_development": { "type": "number", "description": "Research and
development expenses" },
    "interest_expense": { "type": "number", "description": "Interest expense
(absolute value)" },
    "eps_diluted": { "type": "number", "description": "Diluted earnings per
share" },
    "shares_diluted": { "type": "number", "description": "Diluted average shares
outstanding" },
    "total_assets": { "type": "number", "description": "Total assets" },
    "total_liabilities": { "type": "number", "description": "Total liabilities"
},
    "stockholder_equity": { "type": "number", "description": "Total stockholder
equity" },
    "cash": { "type": "number", "description": "Cash and cash equivalents" },
    "short_term_investments": { "type": "number", "description": "Short-term
investments" },
    "long_term_debt": { "type": "number", "description": "Long-term debt" },
    "total_debt": { "type": "number", "description": "Total debt (short + long
term)" },
    "shares_outstanding": { "type": "number", "description": "Shares
outstanding" },
    "cfo": { "type": "number", "description": "Cash flow from operations" },
    "capex": { "type": "number", "description": "Capital expenditures" },
    "fcf": { "type": "number", "description": "Free cash flow" },
    "depreciation": { "type": "number", "description": "Depreciation expense" },
    "stock_based_compensation": { "type": "number", "description": "Stock-based
compensation expense" },
    "source": { "type": "string", "description": "Data source (e.g., 'yahoo')"
},
    "as_of_date": { "type": "string", "format": "date", "description": "Date
when data was fetched" }
  },
  "required": ["ticker", "period_end", "freq", "source", "as_of_date"]
}
```

===== file: entities/PricesHistory =====

json

```
{
  "name": "PricesHistory",
  "type": "object",
  "properties": {
```

```
    "ticker": { "type": "string", "description": "Stock ticker symbol" },
    "date": { "type": "string", "format": "date", "description": "The date of
the price data (YYYY-MM-DD)" },
    "close_adj": { "type": "number", "description": "Adjusted closing price" },
    "open": { "type": "number", "description": "Opening price" },
    "high": { "type": "number", "description": "Highest price of the day" },
    "low": { "type": "number", "description": "Lowest price of the day" },
    "close": { "type": "number", "description": "Closing price (non-adjusted)"
},
    "volume": { "type": "number", "description": "Trading volume for the day" },
    "source": { "type": "string", "description": "The source of the price data
(e.g., yahoo, finnhub)", "default": "yahoo" },
    "as_of_date": { "type": "string", "format": "date", "description": "The date
when the record was last updated" }
  },
  "required": ["ticker", "date", "as_of_date"]
}
```

===== file: entities/Metrics =====

json

```
{
  "name": "Metrics",
  "type": "object",
  "properties": {
    "ticker_symbol": { "type": "string" },
    "ticker": { "type": "string", "description": "Primary ticker identifier" },
    "asOf": { "type": "string", "format": "date" },
    "as_of_date": { "type": "string", "description": "Date of data in YYYY-MM-DD
format" },
    "data_source": { "type": "string", "description": "Source of the data (e.g.,
finnhub, manual)" },
    "price_current": { "type": "number", "description": "The latest stock price"
},
    "eps_ttm": { "type": "number", "description": "The trailing twelve months
Earnings Per Share" },
    "pe_ttm": { "type": "number", "description": "Current P/E TTM ratio" },
    "pe_fwd": { "type": "number", "description": "Forward P/E ratio" },
    "pe_fwd_sector": { "type": "number", "description": "Sector median forward
P/E" },
    "ev_ebitda": { "type": "number", "description": "Enterprise Value to EBITDA
ratio" },
    "ev_ebitda_sector": { "type": "number", "description": "Sector median
EV/EBITDA" },
    "fcf_yield_pct": { "type": "number", "description": "Free cash flow yield
percentage" },
    "peg_5y": { "type": "number", "description": "5-year PEG ratio" },
    "pe_12m": { "type": "number", "description": "P/E TTM 12 months ago" },
    "pe_24m": { "type": "number", "description": "P/E TTM 24 months ago" },
    "pe_36m": { "type": "number", "description": "P/E TTM 36 months ago" },
    "pe_5y_low": { "type": "number", "description": "5-year minimum P/E ratio"
},
    "pe_5y_high": { "type": "number", "description": "5-year maximum P/E ratio"
},
    "pe_5y_median": { "type": "number", "description": "5-year median P/E ratio"
},
    "current_pe": { "type": "number", "description": "Current P/E TTM
(alternative field)" },
    "market_cap": { "type": "number", "description": "Market capitalization" },
    "roic_pct": { "type": "number", "description": "Return on Invested Capital
percentage" },
    "fcf_margin_pct": { "type": "number", "description": "Free cash flow margin
percentage" },
```

```
    "cfo_to_ni": { "type": "number", "description": "Cash from operations to net
income ratio" },
    "fcf_to_ebit": { "type": "number", "description": "Free cash flow to EBIT
ratio" },
    "accruals_ratio": { "type": "number", "description": "Accruals ratio for
quality assessment" },
    "margin_stdev_5y_pct": { "type": "number", "description": "5-year standard
deviation of margins" },
    "buyback_yield_pct": { "type": "number", "description": "Share buyback yield
percentage" },
    "debt_to_equity": { "type": "number", "description": "Total debt to equity
ratio" },
    "netdebt_to_ebitda": { "type": "number", "description": "Net debt to EBITDA
ratio" },
    "interest_coverage_x": { "type": "number", "description": "Interest coverage
ratio (EBIT/Interest)" },
    "eps_cagr_5y_pct": { "type": "number", "description": "5-year EPS compound
annual growth rate" },
    "revenue_cagr_5y_pct": { "type": "number", "description": "5-year revenue
compound annual growth rate" },
    "eps_cagr_3y_pct": { "type": "number", "description": "3-year EPS compound
annual growth rate" },
    "revenue_cagr_3y_pct": { "type": "number", "description": "3-year revenue
compound annual growth rate" },
    "moat_score_0_10": { "type": "number" },
    "recurring_revenue_pct": { "type": "number" },
    "insider_own_pct": { "type": "number" },
    "founder_led_bool": { "type": "boolean" },
    "gm_trend_5y": { "type": "number" },
    "riskdownside_score_0_10": { "type": "number" },
    "beta_5y": { "type": "number", "description": "5-year beta coefficient" },
    "maxdrawdown_5y_pct": { "type": "number", "description": "Maximum 5-year
drawdown percentage" },
    "netcash_to_mktcap_pct": { "type": "number" },
    "sector_cyc_tag": { "type": "string" },
    "macrofit_score_0_10": { "type": "number" },
    "narrative_score_0_10": { "type": "number" },
    "sharecount_change_5y_pct": { "type": "number" },
    "sbc_to_sales_pct": { "type": "number" },
    "proj_growth_rate": { "type": "number" },
    "proj_years": { "type": "number" },
    "proj_target_cagr": { "type": "number" },
    "proj_pe_custom": { "type": "number" },
    "proj_pe_bear_override": { "type": "number" },
    "proj_pe_bull_override": { "type": "number" },
    "proj_pe_mid_override": { "type": "number" },
    "partial_ttm": { "type": "boolean" },
    "revenue_ttm": { "type": "number" },
    "net_income_ttm": { "type": "number" },
    "cfo_ttm": { "type": "number" },
    "capex_ttm": { "type": "number" },
    "ebit_ttm": { "type": "number" },
    "ebitda_ttm": { "type": "number" },
    "total_debt": { "type": "number" },
    "cash": { "type": "number" },
    "equity": { "type": "number" },
    "total_assets": { "type": "number" },
    "shares_out": { "type": "number" },
    "interest_expense_ttm": { "type": "number" }
  },
  "required": ["ticker_symbol"]
}
```

===== file: entities/Ticker =====

json

```
{
  "name": "Ticker",
  "type": "object",
  "properties": {
    "symbol": { "type": "string", "description": "The unique stock ticker
symbol, e.g., 'AAPL'" },
    "exchange": { "type": "string", "description": "The stock exchange, e.g.,
'NASDAQ'" },
    "name": { "type": "string", "description": "The full name of the company" },
    "sector": { "type": "string", "description": "The industry sector of the
company" }
  },
  "required": ["symbol"]
}
```

===== file: entities/LensPreset =====

json

```
{
  "name": "LensPreset",
  "type": "object",
  "properties": {
    "id": { "type": "string" },
    "name": { "type": "string" },
    "valuation": { "type": "number" },
    "quality": { "type": "number" },
    "capitalAllocation": { "type": "number" },
    "growth": { "type": "number" },
    "moat": { "type": "number" },
    "risk": { "type": "number" },
    "macro": { "type": "number" },
    "narrative": { "type": "number" },
    "dilution": { "type": "number" },
    "buyThreshold": { "type": "number", "default": 8.0 },
    "watchThreshold": { "type": "number", "default": 6.5 },
    "mosThreshold": { "type": "number", "default": 10 },
    "scoringHints": { "type": "object" }
  },
  "required": ["id", "name", "valuation", "quality", "capitalAllocation",
"growth", "moat", "risk", "macro", "narrative", "dilution"]
}
```

===== file: src/api/base44Client.js =====

javascript

```
import { createClient } from '@base44/sdk';
import { appParams } from '@/lib/app-params';
const { appId, token, functionsVersion, appBaseUrl } = appParams;
export const base44 = createClient({
  appId,
  token,
  functionsVersion,
  serverUrl: '',
  requiresAuth: false,
  appBaseUrl
});
```

===== file: src/lib/app-params.js =====

javascript

```
const isNode = typeof window === 'undefined';
const windowObj = isNode ? { localStorage: new Map() } : window;
const storage = windowObj.localStorage;
const toSnakeCase = (str) => {
  return str.replace(/([A-Z])/g, '_$1').toLowerCase();
}
const getAppParamValue = (paramName, { defaultValue = undefined, removeFromUrl =
false } = {}) => {
  if (isNode) { return defaultValue; }
  const storageKey = `base44_${toSnakeCase(paramName)}`;
  const urlParams = new URLSearchParams(window.location.search);
  const searchParam = urlParams.get(paramName);
  if (removeFromUrl) {
    urlParams.delete(paramName);
    const newUrl = `${window.location.pathname}${urlParams.toString() ? `?$
{urlParams.toString()}` : ""}${window.location.hash}`;
    window.history.replaceState({}, document.title, newUrl);
  }
  if (searchParam) { storage.setItem(storageKey, searchParam); return
searchParam; }
  if (defaultValue) { storage.setItem(storageKey, defaultValue); return
defaultValue; }
  const storedValue = storage.getItem(storageKey);
  if (storedValue) { return storedValue; }
  return null;
}
const getAppParams = () => {
  if (getAppParamValue("clear_access_token") === 'true') {
    storage.removeItem('base44_access_token');
    storage.removeItem('token');
  }
  return {
    appId: getAppParamValue("app_id", { defaultValue:
import.meta.env.VITE_BASE44_APP_ID }),
    token: getAppParamValue("access_token", { removeFromUrl: true }),
    fromUrl: getAppParamValue("from_url", { defaultValue: window.location.href
}),
    functionsVersion: getAppParamValue("functions_version", { defaultValue:
import.meta.env.VITE_BASE44_FUNCTIONS_VERSION }),
    appBaseUrl: getAppParamValue("app_base_url", { defaultValue:
import.meta.env.VITE_BASE44_APP_BASE_URL }),
  }
}
export const appParams = { ...getAppParams() }
```

===== file: src/lib/utils.js =====

javascript

```
import { clsx } from "clsx"
import { twMerge } from "tailwind-merge"
export function cn(...inputs) {
  return twMerge(clsx(inputs))
}
export const isIframe = window.self !== window.top;
```

===== file: src/lib/AuthContext.jsx =====

jsx

```
import React, { createContext, useState, useContext, useEffect } from 'react';
import { base44 } from '@/api/base44Client';
```

```
import { appParams } from '@/lib/app-params';
import { createAxiosClient } from '@base44/sdk/dist/utils/axios-client';
const AuthContext = createContext();
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [isLoadingAuth, setIsLoadingAuth] = useState(true);
  const [isLoadingPublicSettings, setIsLoadingPublicSettings] = useState(true);
  const [authError, setAuthError] = useState(null);
  const [appPublicSettings, setAppPublicSettings] = useState(null);
  useEffect(() => { checkAppState(); }, []);
  const checkAppState = async () => {
    try {
      setIsLoadingPublicSettings(true);
      setAuthError(null);
      const appClient = createAxiosClient({
        baseURL: `/api/apps/public`,
        headers: { 'X-App-Id': appParams.appId },
        token: appParams.token,
        interceptResponses: true
      });
      try {
        const publicSettings = await appClient.get(`/prod/public-settings/by-id/
${appParams.appId}`);
        setAppPublicSettings(publicSettings);
        if (appParams.token) { await checkUserAuth(); } else {
setIsLoadingAuth(false); setIsAuthenticated(false); }
        setIsLoadingPublicSettings(false);
      } catch (appError) {
        if (appError.status === 403 && appError.data?.extra_data?.reason) {
          const reason = appError.data.extra_data.reason;
          if (reason === 'auth_required') setAuthError({ type: 'auth_required',
message: 'Authentication required' });
          else if (reason === 'user_not_registered') setAuthError({ type:
'user_not_registered', message: 'User not registered for this app' });
          else setAuthError({ type: reason, message: appError.message });
        } else {
          setAuthError({ type: 'unknown', message: appError.message || 'Failed
to load app' });
        }
        setIsLoadingPublicSettings(false);
        setIsLoadingAuth(false);
      }
    } catch (error) {
      setAuthError({ type: 'unknown', message: error.message || 'An unexpected
error occurred' });
      setIsLoadingPublicSettings(false);
      setIsLoadingAuth(false);
    }
  };
  const checkUserAuth = async () => {
    try {
      setIsLoadingAuth(true);
      const currentUser = await base44.auth.me();
      setUser(currentUser);
      setIsAuthenticated(true);
      setIsLoadingAuth(false);
    } catch (error) {
      setIsLoadingAuth(false);
      setIsAuthenticated(false);
      if (error.status === 401 || error.status === 403) {
        setAuthError({ type: 'auth_required', message: 'Authentication required'
});
      }
```

```
    }
  };
  const logout = (shouldRedirect = true) => {
    setUser(null);
    setIsAuthenticated(false);
    if (shouldRedirect) { base44.auth.logout(window.location.href); } else
{ base44.auth.logout(); }
  };
  const navigateToLogin = () =>
{ base44.auth.redirectToLogin(window.location.href); };
  return (
    <AuthContext.Provider value={{ user, isAuthenticated, isLoadingAuth,
isLoadingPublicSettings, authError, appPublicSettings, logout, navigateToLogin,
checkAppState }}>
      {children}
    </AuthContext.Provider>
  );
};
export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth must be used within an AuthProvider');
  return context;
};
```

===== file: src/lib/query-client.js =====

javascript

```
import { QueryClient } from '@tanstack/react-query';
export const queryClientInstance = new QueryClient({
  defaultOptions: {
    queries: {
      refetchOnWindowFocus: false,
      retry: 1,
    },
  },
});
```

===== file: src/components/config/index.jsx =====

javascript

```
export const METRIC_IMPORTANCE_WEIGHTS = {
  pe_fwd: 10, eps_cagr_5y_pct: 10, pe_5y_low: 8, pe_5y_high: 8, roic_pct: 9,
  fcf_margin_pct: 7, revenue_cagr_5y_pct: 7, netdebt_to_ebitda: 6,
interest_coverage_x: 6,
  buyback_yield_pct: 5, peg_5y: 5, fcf_yield_pct: 5, ev_ebitda: 5,
margin_stdev_5y_pct: 4,
  cfo_to_ni: 3, fcf_to_ebit: 3, accruals_ratio: 3, sbc_to_sales_pct: 2,
  sharecount_change_5y_pct: 2, beta_5y: 2, maxdrawdown_5y_pct: 2,
netcash_to_mktcap_pct: 2,
  moat_score_0_10: 3, riskdownside_score_0_10: 3, macrofit_score_0_10: 2,
narrative_score_0_10: 2,
};
export const CRITICAL_METRICS = [
  'pe_fwd', 'eps_cagr_5y_pct', 'pe_5y_low', 'pe_5y_high', 'roic_pct',
'fcf_margin_pct'
];
export const TOTAL_IMPORTANCE_WEIGHT =
Object.values(METRIC_IMPORTANCE_WEIGHTS).reduce((sum, weight) => sum + weight,
0);
export const CONFIDENCE_THRESHOLD = 20;
export const SCORE_DISPLAY_CONFIDENCE_THRESHOLD = 60;
export const DEFAULT_LENS_WEIGHTS = {
```

```
  valuation: 25, quality: 20, capitalAllocation: 15, growth: 20,
  moat: 10, risk: 10, macro: 0, narrative: 0, dilution: 0
};
export const CONFIDENCE_PENALTY_PER_MISSING_CRITICAL_METRIC = 15;
export const MIN_CRITICAL_METRICS_FOR_VALID_SCORE = 4;
// Re-export getSectorMedianDefault from sectorMedians for backward compat
export { getSectorMedianDefault } from './sectorMedians.jsx';
```

===== file: src/components/config/lenses.jsx =====

javascript

```
export const lensRec = {
  "Conservative":       { buy: 8.0, watch: 6.5, mos: 10, scoringHints: {
growthWeights: "45/45/5/5", growthScale: 1.9 } },
  "Value Purist":       { buy: 8.0, watch: 6.5, mos: 15, scoringHints: {
growthWeights: "40/40/10/10", growthScale: 1.8 } },
  "Growth/Momentum":    { buy: 7.8, watch: 6.3, mos: 0,  scoringHints: {
growthWeights: "50/40/8/2", growthScale: 2.1 } },
  "Asymmetry Hunter":   { buy: 7.5, watch: 6.0, mos: 0,  scoringHints: {
growthWeights: "45/45/5/5", growthScale: 1.9 } },
  "Macro-Thematic":     { buy: 7.6, watch: 6.2, mos: 5,  scoringHints: {
growthWeights: "45/45/5/5", growthScale: 1.9 } },
  "Quality Compounder": { buy: 8.2, watch: 6.8, mos: 10, scoringHints: {
growthWeights: "45/45/5/5", growthScale: 1.9 } },
};
```

===== file: src/components/config/metricCriticality.jsx =====

javascript

```
export const metricCriticality = {
  pe_rel: 'Critical', peg: 'Critical', fcf_yield: 'Critical', ev_rel:
'Supportive', pe_hist: 'Contextual',
  roic: 'Critical', fcf_margin: 'Critical', cash_conversion: 'Supportive',
accruals: 'Contextual', margin_stability: 'Contextual',
  buyback: 'Supportive', interest_cover: 'Critical', roiic: 'Supportive',
  eps5y: 'Critical', rev5y: 'Critical', acceleration: 'Supportive', stage_tag:
'Contextual',
  base_moat: 'Critical', recurring: 'Supportive', owner_block: 'Supportive',
  base_risk: 'Critical', net_debt_ebitda: 'Critical', net_cash_mcap:
'Supportive', cyclicality: 'Supportive', beta: 'Contextual', max_drawdown:
'Contextual',
  dilution_risk: 'Supportive',
  macro_fit: 'Contextual', narrative_score: 'Contextual',
};
```

===== file: src/components/config/minimumDataSet.jsx =====

javascript

```
export const MINIMUM_REQUIRED_FIELDS = ['price_current', 'eps_ttm'];
export const PREFERRED_FIELDS = ['revenue_ttm', 'pe_ttm'];
export function checkMFDS(metrics) {
  const missing = [];
  const available = [];
  for (const field of MINIMUM_REQUIRED_FIELDS) {
    const value = metrics[field];
    const isValid = value != null && typeof value === 'number' &&
Number.isFinite(value);
    if (isValid) { available.push(field); } else { missing.push(field); }
  }
  for (const field of PREFERRED_FIELDS) {
    const value = metrics[field];
```

```
    const isValid = value != null && typeof value === 'number' &&
Number.isFinite(value);
    if (isValid) { available.push(field); }
  }
  const isSufficient = missing.length === 0;
  return {
    isSufficient, missing, available,
    coreFieldsCount: MINIMUM_REQUIRED_FIELDS.length,
    availableCoreFields: MINIMUM_REQUIRED_FIELDS.filter(f =>
      metrics[f] != null && typeof metrics[f] === 'number' &&
Number.isFinite(metrics[f])
    ).length
  };
}
```

===== file: src/components/config/sectorMedians.jsx =====

javascript

```
const SECTOR_MEDIANS = {
  "Communication Services": { pe_fwd: 25.0, ev_ebitda: 18.0 },
  "Health Care":            { pe_fwd: 23.0, ev_ebitda: 16.0 },
  "Information Technology": { pe_fwd: 27.0, ev_ebitda: 20.0 },
  "Financials":             { pe_fwd: 13.0, ev_ebitda: 10.0 },
  "Industrials":            { pe_fwd: 18.0, ev_ebitda: 13.0 },
  "Consumer Discretionary": { pe_fwd: 21.0, ev_ebitda: 15.0 },
  "Consumer Staples":       { pe_fwd: 20.0, ev_ebitda: 14.0 },
  "Energy":                 { pe_fwd: 11.0, ev_ebitda:  7.0 },
  "Materials":              { pe_fwd: 16.0, ev_ebitda: 11.0 },
  "Utilities":              { pe_fwd: 17.0, ev_ebitda: 12.0 },
  "Real Estate":            { pe_fwd: 19.0, ev_ebitda: 14.0 },
};
export function getSectorMedianDefault(sector, key) {
  const row = SECTOR_MEDIANS[sector] || null;
  return row && key in row ? row[key] : null;
}
export default SECTOR_MEDIANS;
```

===== file: src/components/lib/derivedFundamentals.jsx =====

javascript

```
const fin = (x) => (typeof x === "number" && Number.isFinite(x)) ? x :
undefined;
export const deriveFundamentals = (inp) => {
  const revenue = fin(inp.revenue_ttm);
  const ebit = fin(inp.ebit_ttm);
  const ebitda = fin(inp.ebitda_ttm);
  const ni = fin(inp.net_income_ttm);
  const cfo = fin(inp.cfo_ttm);
  const capex = fin(inp.capex_ttm);
  const cash = fin(inp.cash);
  const debt = fin(inp.total_debt);
  const equity = fin(inp.equity);
  const assets = fin(inp.total_assets);
  const ie = fin(inp.interest_expense_ttm);
  const mcap = fin(inp.market_cap);
  const fcf_ttm = (cfo !== undefined && capex !== undefined) ? (cfo - capex) :
undefined;
  const fcf_margin_pct = (revenue && fcf_ttm !== undefined && revenue !== 0) ?
(fcf_ttm / revenue) : undefined;
  const fcf_to_ebit = (ebit && fcf_ttm !== undefined && ebit !== 0) ? (fcf_ttm /
ebit) : undefined;
  const taxRate = (revenue && ni && ebit && ebit > 0) ? (1 - ni / ebit) : 0.21;
```

```
  const nopat = (ebit !== undefined) ? ebit * (1 - Math.min(Math.max(taxRate,
0), 0.35)) : undefined;
  const invested = (debt !== undefined && equity !== undefined) ? (debt + equity
- (cash ?? 0)) : undefined;
  const roic_pct = (nopat !== undefined && invested && invested !== 0) ?
(nopat / invested) : undefined;
  const accruals_ratio = (ni !== undefined && cfo !== undefined && assets) ?
((ni - cfo) / assets) : undefined;
  let margin_stdev_5y_pct;
  if (inp.quarterHistory && inp.quarterHistory.length) {
    const recent = inp.quarterHistory.slice(-20);
    const margins = recent
      .map(q => (fin(q.ebit) !== undefined && fin(q.revenue) ? (q.ebit /
q.revenue) : undefined))
      .filter(x => typeof x === "number" && Number.isFinite(x));
    if (margins.length >= 8) {
      const mean = margins.reduce((a,b)=>a+b,0)/margins.length;
      const varc = margins.reduce((a,b)=>a+(b-mean)*(b-mean),0)/margins.length;
      margin_stdev_5y_pct = Math.sqrt(varc);
    }
  }
  const debt_to_equity = (debt !== undefined && equity && equity !== 0) ?
(debt / equity) : undefined;
  const netdebt_to_ebitda = (debt !== undefined && ebitda && ebitda !== 0) ?
(((debt - (cash ?? 0)))/ebitda) : undefined;
  const interest_coverage_x = (ebit !== undefined && ie && ie > 0) ? (ebit / ie)
: undefined;
  let buyback_yield_pct;
  if (mcap && mcap > 0) {
    const soNow = fin(inp.shares_out);
    const soAgo = fin(inp.shares12mAgo);
    if (soNow && soAgo && soNow > 0) {
      buyback_yield_pct = Math.max(0, (soAgo - soNow) / soNow);
    }
  }
  return { fcf_ttm, fcf_margin_pct, fcf_to_ebit, roic_pct, accruals_ratio,
margin_stdev_5y_pct, debt_to_equity, netdebt_to_ebitda, interest_coverage_x,
buyback_yield_pct };
};
```

===== file: src/components/lib/peHistory.jsx =====

javascript

```
const isFin = (x) => typeof x === "number" && Number.isFinite(x);
export const buildDailyTTM = (ttm, days) => {
  const out = {};
  if (!ttm?.length || !days?.length) return out;
  let i = 0;
  for (const d of days) {
    while (i + 1 < ttm.length && ttm[i + 1].date <= d) i++;
    out[d] = ttm[i].eps_ttm;
  }
  return out;
};
const onOrBefore = (targetISO, days) => {
  let lo = 0, hi = days.length - 1, ans = -1;
  while (lo <= hi) {
    const mid = (lo + hi) >> 1;
    if (days[mid] <= targetISO) { ans = mid; lo = mid + 1; } else hi = mid - 1;
  }
  return ans;
};
const monthsAgoISO = (asOfISO, months) => {
```

```
  const d = new Date(asOfISO + "T00:00:00Z");
  d.setMonth(d.getMonth() - months);
  return d.toISOString().slice(0,10);
};
export const peAtMonthsAgo = (prices, epsDaily, months, asOfISO) => {
  if (!prices?.length) return undefined;
  const days = prices.map(p => p.date);
  const idxAsOf = onOrBefore(asOfISO, days);
  if (idxAsOf < 0) return undefined;
  const anchor = monthsAgoISO(asOfISO, months);
  const idx = onOrBefore(anchor, days);
  if (idx < 0) return undefined;
  const price = prices[idx].close;
  const eps = epsDaily[days[idx]];
  return (isFin(price) && isFin(eps) && eps > 0) ? price / eps : undefined;
};
export const peSeries5Y = (prices, epsDaily, asOfISO) => {
  if (!prices?.length) return [];
  const cutoff = new Date(asOfISO + "T00:00:00Z");
  cutoff.setFullYear(cutoff.getFullYear() - 5);
  const cutISO = cutoff.toISOString().slice(0,10);
  const out = [];
  for (const p of prices) {
    if (p.date < cutISO || p.date > asOfISO) continue;
    const e = epsDaily[p.date];
    if (isFin(p.close) && isFin(e) && e > 0) out.push(p.close / e);
  }
  return out;
};
export const percentile = (v, p) => {
  if (!v?.length) return undefined;
  const a = [...v].sort((x,y)=>x-y);
  const i = Math.floor((a.length - 1) * p);
  return a[i];
};
```

===== file: src/components/utils/num.jsx =====

javascript

```
export const clamp = (x, lo, hi) => Math.max(lo, Math.min(hi, x));
export const clamp01 = (x) => clamp(x, 0, 1);
export const nz = (x) => (x === null || x === undefined || Number.isNaN(+x)) ?
null : +x;
export const toPoints = (v) => {
  if (v === null || v === undefined || v === '') return null;
  const s = ('' + v).trim();
  if (s.endsWith('%')) return +s.slice(0, -1);
  const n = +s;
  if (Number.isNaN(n)) return null;
  if (n > -1 && n < 1 && n !== 0) return n * 100;
  return n;
};
export const toNumber = (v) => {
  if (v === null || v === undefined || v === '') return null;
  const n = +v;
  return Number.isNaN(n) ? null : n;
};
export function fmt0(x) { return (x == null || Number.isNaN(+x)) ? "—" :
String(x); }
export function fmt1(x) { return (x == null || Number.isNaN(+x)) ? "—" :
(+x).toFixed(1); }
export function fmt2(x) { return (x == null || Number.isNaN(+x)) ? "—" :
(+x).toFixed(2); }
```

===== file: src/components/utils/normalizeSymbol.jsx =====

javascript

```
export function normalizeSymbol(s) {
  return (s || '').toString().trim().toUpperCase();
}
export function normalizeExchange(e) {
  return (e || '').toString().trim().toUpperCase();
}
```

===== file: src/components/utils/peBand.jsx =====

javascript

```
export function cleanBand(peLow, peHigh, peTTM) {
  const low = isFinite(peLow) ? Number(peLow) : null;
  const high = isFinite(peHigh) ? Number(peHigh) : null;
  const ttm = isFinite(peTTM) ? Number(peTTM) : null;
  if (low != null && (low <= 0 || low > 200)) return {low:null, high:null, ttm};
  if (high != null && (high <= 0 || high > 400)) return {low:null, high:null,
ttm};
  if (low != null && high != null && low > high) return {low:null, high:null,
ttm};
  return {low, high, ttm};
}
export function bandMid(low, high) {
  return (low!=null && high!=null
```